This problem set is due *by 11:59pm* on **Wednesday, April 23**. It should be submitted through Canvas. All submissions should be typeset using Latex.

Problem 1 (Nonlinear Function Approximation) *In lecture, we discussed nonlinear function approximation. In this problem, we will explore nonlinear parameter estimation for a fixed-wing UAV.*

a) ***(2 points)*** *Assume we want to estimate l_w and S_e in the fixed-wing UAV dynamics model. Can this be done via linear function approximation? Why or why not?*

b) ***(2 points)*** *Examine the code in* `glider_nonlinear_parameter_estimation.m`*. Compute linear control gains using LQR for a reduced state-space, where the x-position state is ignored. Implement this control law in the control loop.*

c) ***(3 points)*** *Complete the* `pglider_7d_augmented` *function. First, set the unknown parameters using the augmented state. Then, compute the state from the augmented state to pass into the UAV dynamics. Then construct the derivative of the augmented state using the state derivatives from the UAV dynamics.*

d) ***(5 points)*** *Complete the* `extended_kalman_filter` *function. First estimate the next state and covariance using the linearized process model. Then, generate the linearization of the measurement model and compute the measurement and covariance residual. Finally compute the Kalman gains and compute the covariance update.*

e) ***(3 points)*** *Run the simulation. Submit your code and plot of the estimated parameters.*

Problem 2 (Model-Based Reinforcement Learning) *In this problem, we will explore a type of model-based reinforcement, closely related to guided policy search. We will also make use of linear function approximation.*

a) ***(5 points)*** *Linear least-squares function approximation often minimizes a cost function of the form $J = \frac{1}{2}e^T e$, where $e = y - \Phi(x)\theta$, θ are the linear parameters (i.e., weights), and $\Phi(x)$ is a set of nonlinear basis functions. However, it is often more advantageous to use "regularized" least-squares when noise is present. That is, minimize a cost function of the form $J = \frac{1}{2}e^T e + \frac{1}{2}\gamma\theta^T\theta$. Derive an expression for $\hat{\theta}$, or the least-squares estimate, for the regularized least-squares case. Describe why minimizing this cost function may produce an estimate $\hat{\theta}$ which is more robust.*

b) ***(5 points)*** *We are going to use regularized least-squares to fit the lift-coefficient for a flat-plate glider. Explore the code in* `fit_coeffs.m`*. Complete the function* `gaussian` *to evaluate a set of N radial basis function defined by covariance Σ and centers defined by μ.*

c) ***(5 points)*** *Use your solution from part (a) to compute the parameter estimate using regularized least squares where* $\gamma = 0.1$.

d) ***(5 points)*** *Now we will explore the use of this linear function approximator in a model-based reinforcement learning approach for the perching glider example. In this problem, we will leverage radial basis function weights learned from real data. Our simulator will be based on a RBF NN where the input space is angle-of-attack, elevator angle, and velocity. However, our controller will only be using a RBF NN with a two-dimensional input space: angle-of-attack and elevator angle. In this way, our learning-based controller can not fully represent the simulated dynamics model. Using your answers from parts (a-c), fill in the* `gaussian` *function to evaluate the RBF NN. Then complete the* `least_squares` *function. Note: The dynamics function returns* f *and* g *where* $\dot{x} = f(x, u) + g(x, u)\theta$*, and* θ *are the weights of the RBF NN. In the main iteration loop, also fill in the state derivatives, which are the training targets.*

e) ***(3 points)*** *We will use iLQR to generate the policy using this learned dynamics model. As we observed with Guided Policy Search (GPS), keeping the new policy close to the policy generated in the previous iteration is important. Ideally, we want to only update our policy in regions where we have data and the model is valid. Simpler cost structures can also be used than the one proposed for GPS. Design a simple quadratic cost to keep iLQR policies close to the policies generated on previous iterations. Also implement the control law in the simulation loop.*

f) ***(2 points)*** *Run your code. Generate position plots of your final trajectory after 20 iterations. Also plot the final cost on each learning iteration. Submit your code and plots.*